

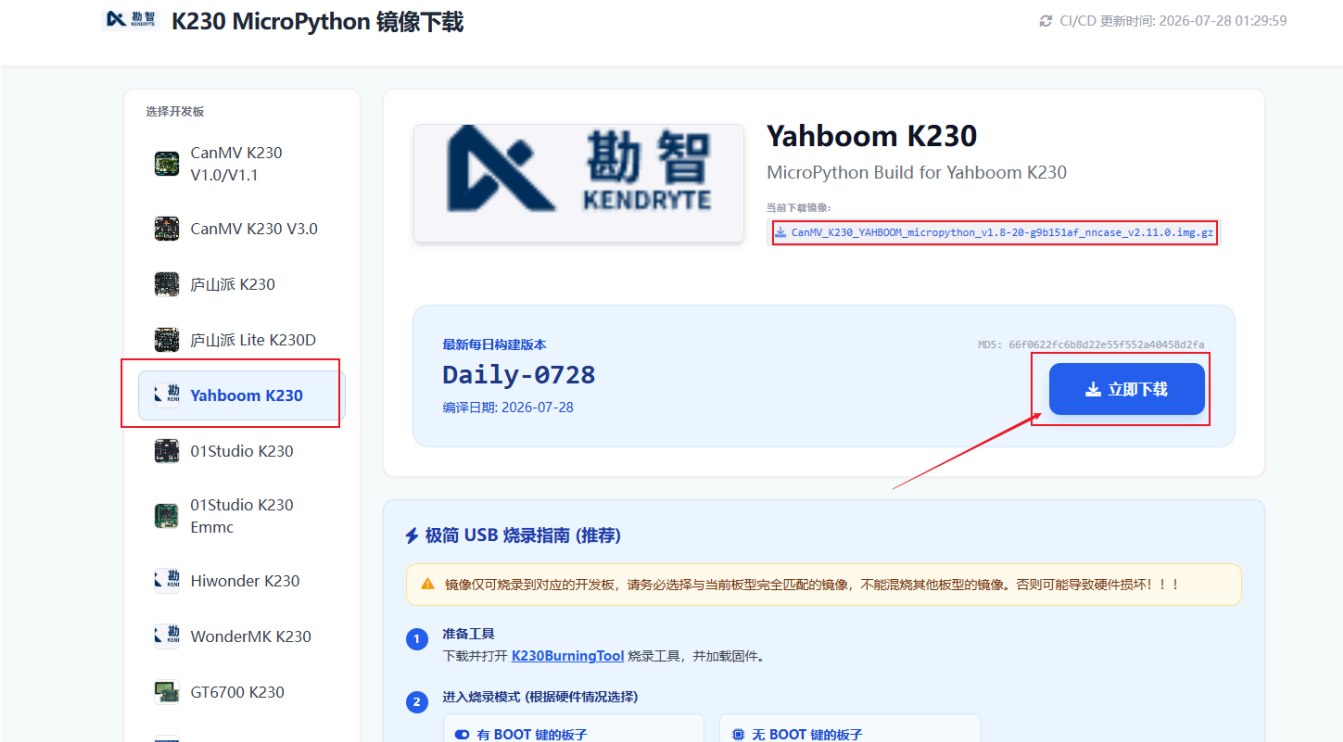
Channel Operation and Statistics

Channel Operation and Statistics

- Preface
- Example Results
- Theory
 - Channel Split and Merge
 - Mean and Extreme Values
- Code Overview
 - Initialize Camera and Display Module
 - Set Operation Mode
 - Split and Display Single Channel
 - Swap Red and Blue Channels
 - Calculate Mean and Extreme Values
 - Draw Statistical Information
 - Resource Release
- Parameter Adjustment Notes

Preface

This tutorial applies to firmware version: CanMV_K230_YAHBOOM_micropython_v1.8-13. You can download it from the Kendryte official website at https://download.kendryte.com/developer/releases/canmv_k230_micropython/daily_build/



Example Results

Run the example code for this section: [16.camera_split_merge.py](#).

This section demonstrates B, G, R channel splitting, channel reordering, three-channel mean statistics, and locating the brightest and darkest points in grayscale images. The default mode draws statistical values and extreme positions on the original image.



Theory

Channel Split and Merge

Color image arrays have shape HxWx3. `split()` splits it into three HxW single-channel arrays, and `merge()` recombines them in a given order. Swapping the first and last channels causes red and blue visual effects to interchange.

At 640x480, each single-channel 8-bit array occupies approximately 300KB, and a three-channel array occupies approximately 900KB. One `split()` followed by `merge()` creates multiple temporary arrays, requiring attention to memory and frame rate.

Mean and Extreme Values

- `mean()` returns the average value of each channel, useful for rough judgment of overall color and brightness.
- `minMaxLoc()` returns the minimum, maximum values and their coordinates for a single-channel array.

Extreme values are easily affected by dead pixels, reflections, and pure black edges. Actual projects usually first define ROI or apply smoothing.

Code Overview

Initialize Camera and Display Module

```
W, H = 640, 480
sensor = Sensor(width=1280, height=960, fps=90)
sensor.reset()
sensor.set_framesize(width=W, height=H)
sensor.set_pixformat(Sensor.RGB888)
Display.init(Display.ST7701, width=640, height=480, to_ide=True)
sensor.run()
time.sleep(0.5)
```

Set Operation Mode

```
mode = 4
```

\$mode\$	Function
0	Display B channel
1	Display G channel
2	Display R channel
3	Swap B, R channels
4	Original image with mean and extreme values, source default

Split and Display Single Channel

```
if mode <= 2:
    channels = cv2.split(img_np)
    out = channels[mode]
    img_out = image.Image(
        W, H, image.GRAYSCALE,
        alloc=image.ALLOC_REF,
        data=out
    )
    Display.show_image(img_out)
```

Single-channel arrays are packaged as grayscale images for display. Higher brightness indicates larger values in the corresponding color channel.

Swap Red and Blue Channels

```
elif mode == 3:
    channels = cv2.split(img_np)
    img_np[:, :, :] = cv2.merge([
        channels[2],
        channels[1],
        channels[0]
    ])
    Display.show_image(img)
```

The merge order changes from B, G, R to R, G, B. This mode holds three split channels and one merge result simultaneously, with higher memory pressure.

Calculate Mean and Extreme Values

```
else:
    gray = cv2.cvtColor(img_np, cv2.COLOR_BGR2GRAY)
    average = cv2.mean(img_np)
    minimum, maximum, min_location, max_location = cv2.minMaxLoc(gray)
```

The first three values of average are used to display B, G, R averages. minMaxLoc() only accepts grayscale images and returns brightest and darkest positions.

Draw Statistical Information

```
cv2.putText(
    img_np,
    "B={:.0f}G={:.0f}R={:.0f}".format(
        average[0], average[1], average[2]
    ),
    (5, 15), cv2.FONT_HERSHEY_SIMPLEX,
    0.4, (255, 255, 0), 1
)

cv2.drawMarker(
    img_np, max_location,
    (0, 255, 0),
    cv2.MARKER_CROSS, 15, 1
)

cv2.drawMarker(
    img_np, min_location,
    (255, 0, 0),
    cv2.MARKER_CROSS, 15, 1
)
```

Green cross marks the brightest point; blue cross marks the darkest point.

Resource Release

```
finally:  
    sensor.stop()  
    Display.deinit()  
    os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)  
    time.sleep_ms(100)
```

Parameter Adjustment Notes

- **Modes 0~2:** Three channels are created every frame even if only one is displayed. If only a fixed channel is needed, evaluate whether direct slicing meets subsequent API and memory layout requirements.
- **Mode 3 memory:** At 640x480, additional allocation can reach approximately 1.8MB. Observe frame rate and frame capture stability.
- **Mean meaning:** Mean is dominated by large background areas. For target color statistics, first crop ROI or use a mask.
- **Extreme value noise immunity:** A single reflection point may become the maximum; a single black edge may become the minimum. Consider blurring or ignoring edge areas first.
- **Channel order:** The tutorial explains in B, G, R order based on the current example. When connecting to other data sources, first confirm its actual channel layout.